

Champion Document — Pickups, Experience & the Upgrade Draft

Name: Michael Jones **Feature:** F4 **Mark:** ____ / 30

Feature scope: Drops, magnet, XP curve, level-up pause, three-card draft, and run loot.

This Champion describes the reward loop: enemies and other reward sources create pickups; the player collects them; experience advances the player along an XP curve; level-ups pause gameplay and present three upgrade cards; and run loot is recorded for the current run.

1. Brief Introduction

My feature for our game is pickups, experience, and the upgrade draft. When an eligible enemy is defeated, the reward system rolls the configured drop table and creates collectible pickups in the room. Pickups may include experience fragments, ember shards, healing items, or other run resources defined by the item table.

A pickup can be collected by direct contact or by the player's magnet radius. Experience pickups are added to the player's current XP total, and the XP curve determines the amount required for the next level. When the player reaches the required amount, the game enters a level-up pause: gameplay stops, a three-card upgrade draft is generated, and the player selects one card before play resumes.

My feature also owns run-loot accounting. It records pickups collected during the current run and transfers only the configured persistent rewards to the run summary when the run ends. It does not resolve enemy damage or decide the contents of individual upgrade effects; those belong to Combat and Upgrade Definition data respectively.

2. Use Case Diagram with Scenario

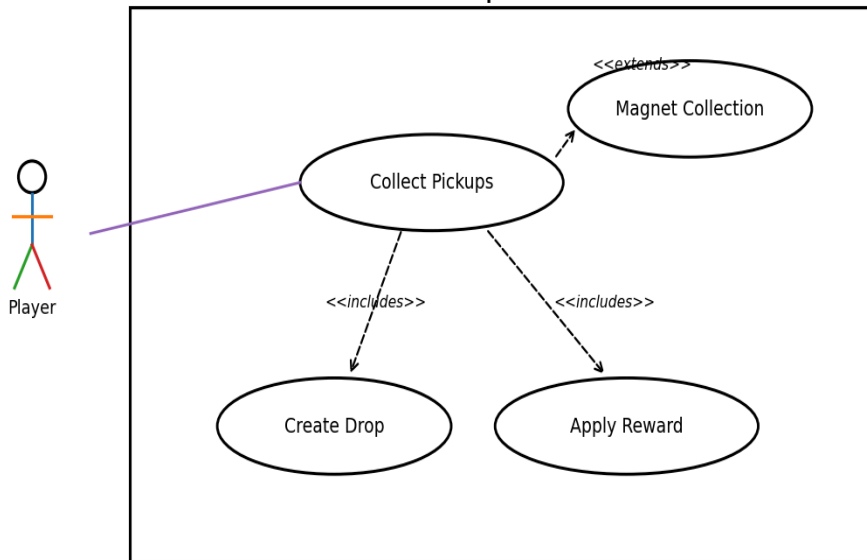
Use Case Diagrams

The following diagrams show the actors, main use cases, included sub-functions, and exception/extension cases for this feature.

Scenarios

Scenario 1: Create and Collect Pickups

I Got Transported to Another World and Got Married to a Vampire Shrimp But His Vampire Family Doesn't Approve Of Our Marriage
— Pickup Collection



Name: Create and Collect Pickups

Summary: When a reward source produces a drop, the feature creates valid pickups and allows the player to collect them directly or through the magnet.

Actors: Enemy/Reward Source; Player

Preconditions: The room is active; the pickup table is loaded; the player exists; and the reward source has produced a valid reward event.

Basic sequence:

Step 1: The reward source sends a drop event containing the source position, reward category, and run seed.

Step 2: The drop table selects the pickup type and quantity using the configured weighted rules.

Step 3: The feature creates each pickup at a valid position near the source and registers it with Pickup Runtime Data.

Step 4: The player enters a pickup's contact radius or magnet radius.

Step 5: The pickup moves toward the player if magnet collection is active, or is collected immediately on contact.

Step 6: The feature validates that the pickup is uncollected, applies its reward, marks it collected, and updates run-loot accounting.

Exceptions:

Step 2a: The selected reward is disabled for the current room or run state. The reward is rerolled up to 5 times; if no valid reward is found, no pickup is created and the event is logged.

Step 3a: A candidate drop position is invalid or occupied. A new position is generated; after 10 failed attempts, the pickup is placed at the source position with collision-safe fallback rules.

Step 5a: The player leaves magnet range before collection. The pickup stops homing and returns to its idle state.

Step 6a: The pickup has already been collected. The event is ignored and the reward is not applied a second time.

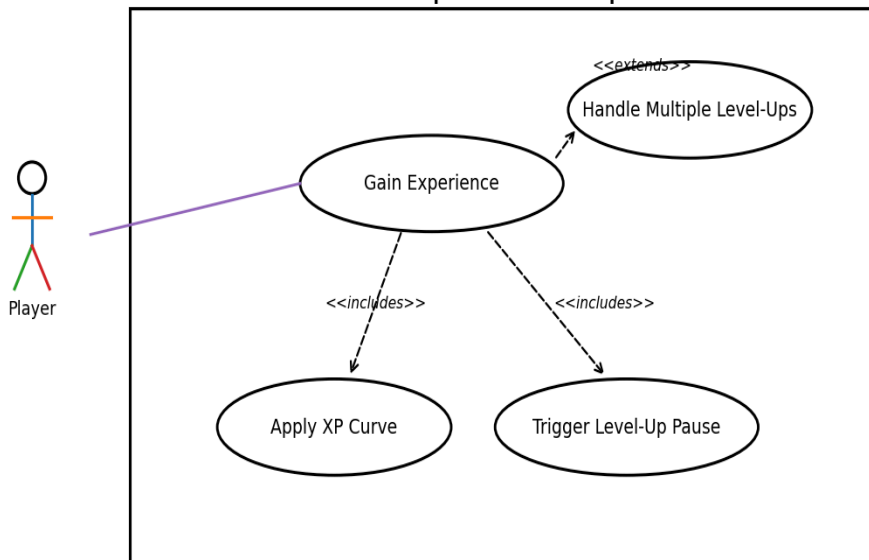
Post conditions: Every created pickup is registered, every collected pickup applies its reward once, and the run-loot ledger reflects successful collection.

Priority: 1 (must have)

ID: PC1

Scenario 2: Gain Experience and Trigger Level-Up

I Got Transported to Another World and Got Married to a Vampire Shrimp But His Vampire Family Doesn't Approve Of Our Marriage
— Experience & Level-Up



Name: Gain Experience and Trigger Level-Up

Summary: Collected experience increases the player's XP total. When the level threshold is reached, the game pauses and begins the upgrade draft.

Actors: Player; Level Runtime

Preconditions: The player is alive and the XP curve table is loaded.

Basic sequence:

Step 1: The player collects an experience pickup.

Step 2: The feature adds the pickup's XP value to the player's current XP total.

Step 3: The feature compares current XP against the threshold for the player's next level.

Step 4: If the threshold is reached, the feature increases the player's level and subtracts or carries forward excess XP according to the configured XP policy.

Step 5: The feature requests a gameplay pause and opens the upgrade draft.

Step 6: The draft system generates three eligible, non-duplicate upgrade cards and displays them to the player.

Exceptions:

Step 2a: The player reaches multiple thresholds from one pickup. The feature processes each threshold in order and creates one draft pause per level, preserving excess XP.

Step 3a: XP is received while the level-up pause is active. The XP is queued and applied after the current draft is completed; it cannot alter the three cards already displayed.

Step 6a: Fewer than three eligible upgrades exist. The draft displays all remaining eligible cards and fills any missing slot with a clearly marked reroll-disabled placeholder.

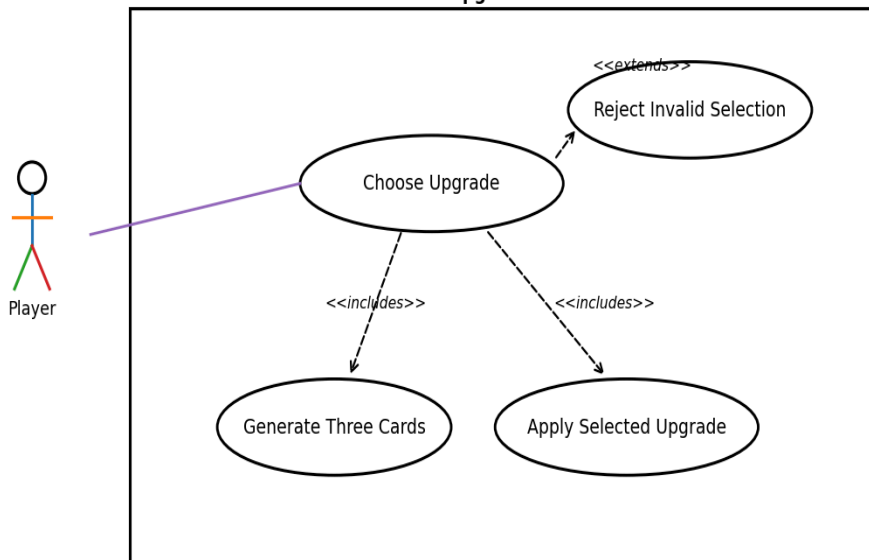
Post conditions: The player's XP and level are consistent with the XP curve, and a valid draft is active whenever a level-up requires a choice.

Priority: 1 (must have)

ID: XP1

Scenario 3: Choose an Upgrade and Resume the Run

**I Got Transported to Another World and Got Married to a Vampire Shrimp But His Vampire Family Doesn't Approve Of Our Marriage
— Upgrade Draft**



Name: Choose an Upgrade and Resume the Run

Summary: The player selects one card from a three-card draft. The selected upgrade is applied once, the draft closes, and gameplay resumes.

Actors: Player; Upgrade Draft System

Preconditions: A level-up pause is active and the draft contains at least one selectable card.

Basic sequence:

Step 1: The draft displays three card slots with each card's name, description, and eligibility state.

Step 2: The player selects one eligible card.

Step 3: The feature validates that the card is still eligible and that no card has already been selected for this draft.

Step 4: The selected upgrade is added to the player's run build and its effect is applied.

Step 5: The selected card is recorded in the run-loot and upgrade history.

Step 6: The draft closes and gameplay resumes.

Exceptions:

Step 2a: The player selects a locked, duplicate, or placeholder card. The selection is rejected and the pause remains active.

Step 3a: The selected card becomes invalid because of a state change. The feature removes it from selection and generates a replacement if the draft has not been confirmed.

Step 4a: Applying the upgrade fails validation. No partial effect is committed, an error is logged, and the player remains in the level-up pause.

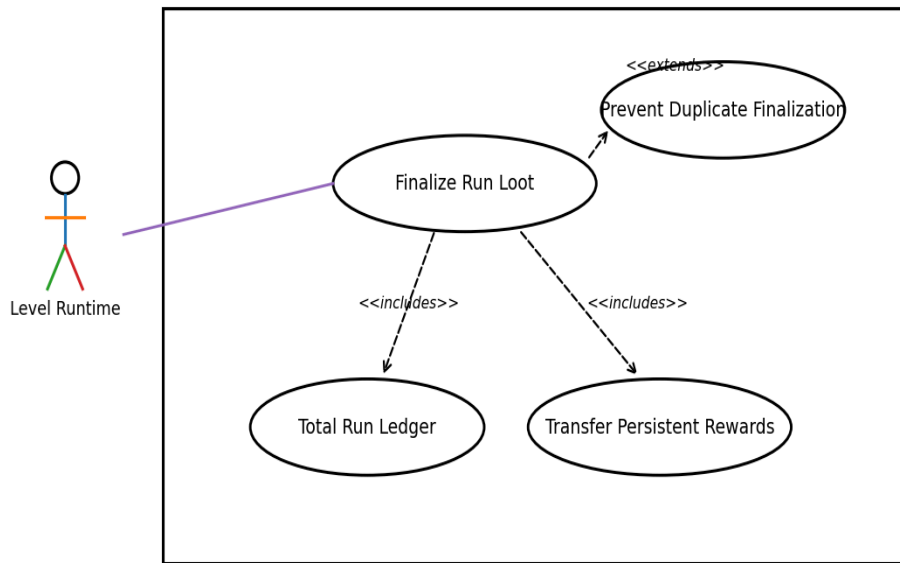
Post conditions: Exactly one upgrade is selected for the draft, its effect is committed once, and the game resumes only after successful confirmation.

Priority: 1 (must have)

ID: UD1

Scenario 4: Finalize Run Loot

I Got Transported to Another World and Got Married to a Vampire Shrimp But His Vampire Family Doesn't Approve Of Our Marriage
— Run Loot



Name: Finalize Run Loot

Summary: At run end, the feature totals the current run's collected resources and sends the configured persistent rewards to the run summary.

Actors: Level Runtime; Player

Preconditions: The run has entered a valid end state and the run-loot ledger is available.

Basic sequence:

Step 1: Level Runtime sends a run-ended event.

Step 2: The feature closes any active pickup collection operations and rejects new collection events.

Step 3: The ledger totals collected resources, selected upgrades, and other configured run statistics.

Step 4: The persistence policy filters rewards that are eligible to leave the run.

Step 5: The feature sends the filtered summary to the profile or run-summary system.

Step 6: The feature marks the run ledger finalized and prevents duplicate finalization.

Exceptions:

Step 2a: A pickup collection event arrives after run end. The event is ignored and no reward is added.

Step 4a: A reward is not persistent by policy. It remains in the run summary but is not transferred to the permanent profile.

Step 6a: Finalization is requested twice. The second request returns the existing finalized summary without adding rewards again.

Post conditions: The run summary is complete, persistent rewards are transferred at most once, and the finalized ledger is immutable.

Priority: 2 (should have)

ID: RL1

3. Data Flow Diagrams

The context diagram treats F4 as a single system process and shows only the external systems and data crossing the feature boundary. Reward sources send drop events; the player supplies collection and draft-selection actions; Level Runtime receives pause/resume requests; the XP Curve and Upgrade Definition tables supply configuration data; and the Run Summary/Profile system receives finalized persistent rewards.

Context Diagram

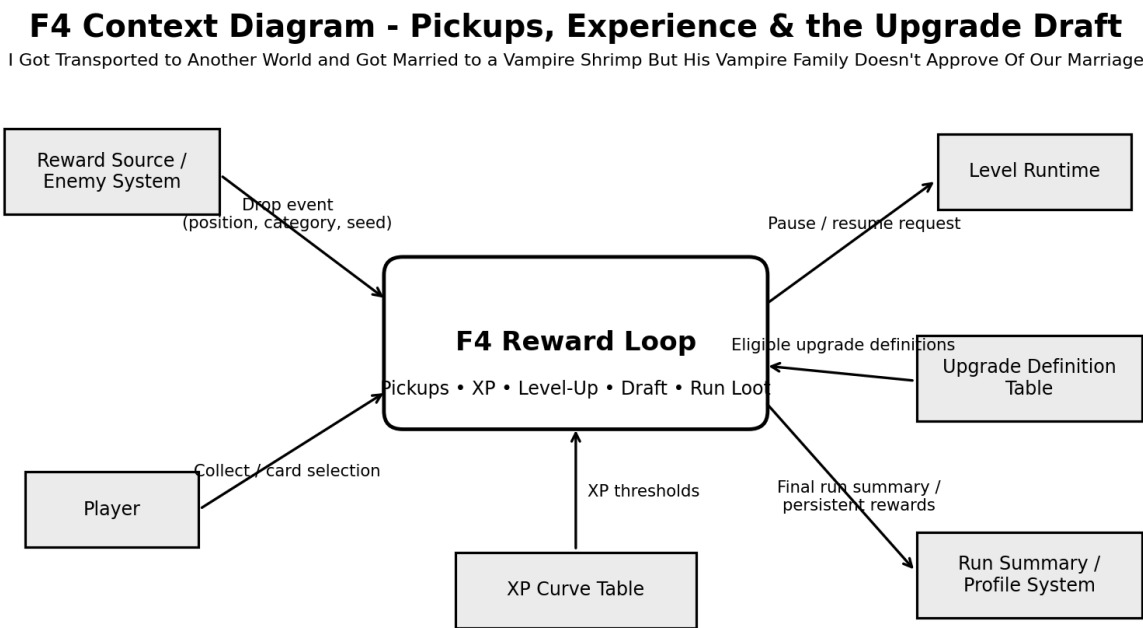
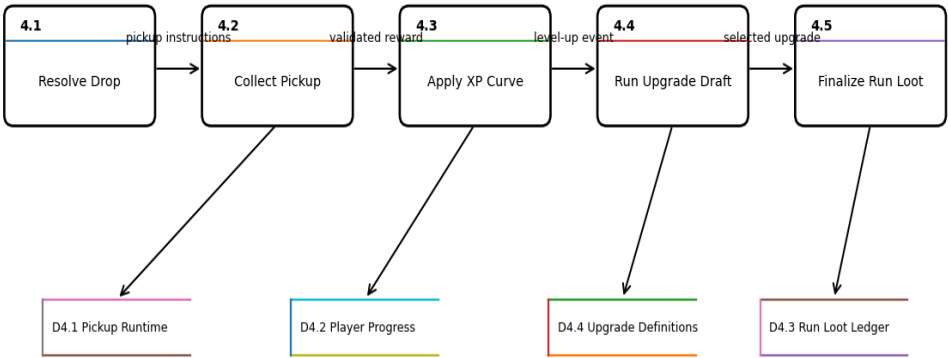


Diagram 0 shows the whole game with the F4 processes highlighted. External entities are the Player, Reward Source, Level Runtime, Upgrade Definition Table, XP Curve Table, and Run Summary/Profile System.

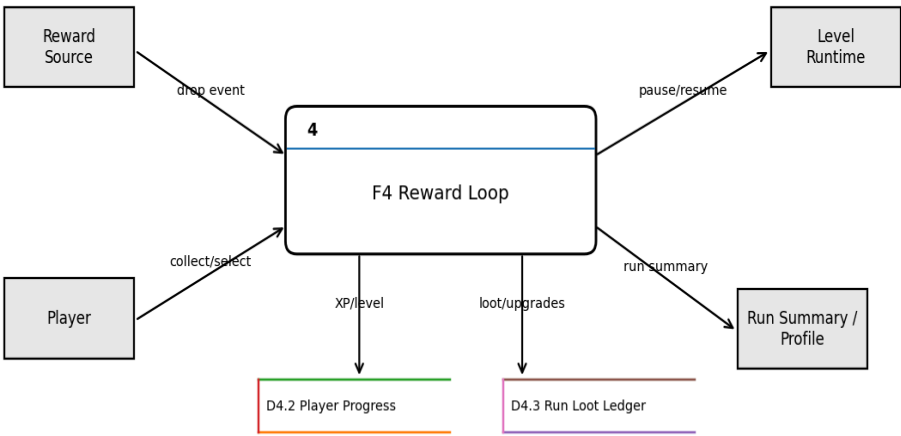
Diagram 0 - I Got Transported to Another World and Got Married to a Vampire Shrimp But His Vampire Family Doesn't Approve Of Our Marriage Reward Loop Context

I Got Transported to Another World and Got Married to a Vampire Shrimp But His Vampire Family Doesn't Approve Of Our Marriage – F4 Subprocess Data Flow Diagram



Feature Subprocess Diagram - F4 Reward Loop

I Got Transported to Another World and Got Married to a Vampire Shrimp But His Vampire Family Doesn't Approve Of Our Marriage – Reward Loop Context DFD



Recommended Gane–Sarson process breakdown:

Process	Name	Inputs	Outputs
4.1	Resolve Drop	Drop event, drop table, seed	Pickup type, quantity, spawn instructions
4.2	Collect Pickup	Player position, contact/magnet event, pickup data	Validated reward event, updated ledger
4.3	Apply XP Curve	XP reward, current XP, curve table	New XP, level-up event

4.4	Run Upgrade Draft	Level-up event, upgrade table, build state	Three cards, selected upgrade, resume event
4.5	Finalize Run Loot	Run-ended event, ledger, persistence policy	Run summary, persistent reward transfer

Data stores:

- D4.1 Pickup Runtime Data: active pickups, positions, collected flags, and pickup identifiers.
- D4.2 Player Progress Data: current XP, level, and XP carry-over.
- D4.3 Run Loot Ledger: collected resources, selected upgrades, and finalization status.
- D4.4 Upgrade Definition Table: card requirements, rarity, tags, and effect references.
- D4.5 XP Curve Table: level thresholds and maximum level rules.

Process Description — 4.4 Run Upgrade Draft

Decision-tree process description:

1. Receive a valid level-up event.
2. Freeze gameplay and mark the draft state ACTIVE.
3. Query the upgrade definition table for cards eligible for the player's current build and run state.
4. Remove duplicates and cards that violate mutually exclusive requirements.
5. Randomly select up to three cards using the configured draft weights.
6. If fewer than three cards are eligible, display the available cards and disabled placeholders.
7. Wait for player selection.
8. Validate the selected card, commit its effect atomically, and record the choice.
9. Close the draft, mark the draft COMPLETE, and resume gameplay.

Reward Loop Pseudocode

The following pseudocode describes the primitive control logic connecting drops, magnet collection, XP thresholds, queued level-ups, the three-card draft, and run-loot finalization.

```

REWARD_LOOP(event):
  IF event.type == REWARD_SOURCE_DEFEATED:
    reward = ResolveDrop(event.category, event.runSeed)
    IF reward is valid:
      position = FindValidDropPosition(event.position, maxAttempts = 10)
      pickup = CreatePickup(reward.type, reward.quantity, position)
      RegisterPickup(pickup)

  FOR EACH active pickup:
    IF PlayerInsideMagnetRadius(pickup):
      MovePickupTowardPlayer(pickup)
    ELSE:
      SetPickupIdle(pickup)

  IF PlayerInsideCollectionRadius(pickup) AND pickup.collected == false:
    pickup.collected = true
    rewardResult = ApplyPickupReward(pickup)
    RecordRunLoot(rewardResult)

  IF rewardResult contains XP:
    AddXP(rewardResult.xp)

```

```

        WHILE currentXP >= XPRequiredForNextLevel():
            currentXP = currentXP - XPRequiredForNextLevel()
            level = level + 1
            QueueLevelUpDraft(level)

    IF LevelUpDraftQueued() AND DraftIsActive():
        PauseGameplay()
        eligible = GetEligibleUpgradeCards(playerBuild)
        choices = SelectUpToThreeUniqueCards(eligible)
        DisplayDraft(choices, disabledPlaceholdersIfNeeded = true)

    IF DraftIsActive() AND PlayerSelectsCard(card):
        IF CardIsEligible(card) AND DraftHasNoCommittedChoice():
            ApplyUpgradeAtomically(card)
            RecordSelectedUpgrade(card)
            CloseDraft()

            IF AnotherLevelUpDraftIsQueued():
                OpenNextQueuedDraft()
            ELSE:
                ApplyQueuedXP()
                ResumeGameplay()

    IF RunEnded():
        RejectNewPickupCollections()
        summary = FinalizeRunLootOnce()
        persistentRewards = FilterPersistentRewards(summary)
        SendToRunSummaryProfile(persistentRewards)

```

4. Acceptance Tests

The feature contains deterministic rules (XP thresholds, collection validation, draft eligibility, and finalization) and bounded random elements (drop type, pickup quantity, and card selection). Tests run on the testing compile path.

Pickup collection test

Run 1,000 scripted collection attempts: 500 by direct contact and 500 by magnet. Pass requires 1,000 successful valid collections, zero duplicate rewards, and zero rewards applied to pickups marked collected.

Magnet behavior test

Run 500 magnet trials with the player entering and leaving magnet range. Pass requires magnet movement to begin within 0.20 seconds of entering range, movement to stop within 0.20 seconds of leaving range, and no pickup to be collected outside the configured collection radius.

XP curve test

Test every configured level threshold with exact-threshold, one-point-below, and one-point-above inputs. Pass requires zero early level-ups, every exact threshold to level up, and all excess XP to be preserved according to the configured carry-over rule.

Level-up pause test

Trigger 500 level-ups using scripted XP events. Pass requires gameplay to be paused within 0.20 seconds of the level-up event, no enemy movement or damage simulation during the pause, and gameplay to resume within 0.50 seconds after a valid selection.

Three-card draft test

Generate 1,000 drafts across valid build states. Pass requires every draft to contain exactly three visible slots, no duplicate selectable cards within a draft, and every selectable card to satisfy its eligibility rules. If fewer than three cards are eligible, all unavailable slots must be disabled placeholders.

Upgrade application test

Perform 500 valid selections and 200 invalid selection attempts. Pass requires every valid selection to apply exactly once, all invalid selections to apply zero effects, and each draft to close only after a valid committed selection.

Run loot finalization test

Finalize 500 completed runs, including 100 repeated-finalization attempts. Pass requires every run to produce one summary, no persistent reward to be transferred more than once, and all nonpersistent resources to remain excluded from the permanent profile.

Example output test check analysis

Test	Runs	Observed	Bound	Pass?
Duplicate collection	1,000 attempts	0 duplicates	0 allowed	Yes
Magnet start latency	500 trials	0.11 s mean	≤ 0.20 s	Yes
XP threshold	All levels	1 early level-up	0 allowed	No
Draft uniqueness	1,000 drafts	0 duplicate cards	0 allowed	Yes
Invalid upgrade selection	200 attempts	0 effects applied	0 allowed	Yes
Run finalization	500 runs + 100 repeats	0 duplicate transfers	0 allowed	Yes

5. Timeline

Task	Duration (hours)	Predecessor task(s)
1. Pickup and reward data design	3	—
2. Drop table and spawn logic	4	1
3. Pickup collision and magnet behavior	4	2
4. XP curve and progression state	4	1
5. Level-up pause state machine	3	4
6. Upgrade card eligibility and draft	5	5
7. Upgrade selection and application interface	4	6
8. Run-loot ledger and finalization	3	1, 7
9. Integration testing	5	3, 4, 7, 8
10. Documentation and diagrams	3	2, 6, 8
11. UI and pickup/card presentation	5	2, 6

PERT summary: The estimated critical path is $1 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$, totaling 28 hours. Pickup presentation and documentation can proceed in parallel after their predecessor tasks are complete. These estimates assume that Combat, Level Runtime, and the player profile interfaces already exist and only need integration points.

Gantt timeline: Schedule Tasks 1 and 4 first. Begin Tasks 2 and 5 after their respective data dependencies are available. Tasks 3, 6, and 11 can overlap where their interfaces are stable. Reserve Task 9 for the final integrated build.

6. AI Question Log — Summary

I used AI assistance to structure the Champion document around the sample's feature format. The main design decisions were to make every exception reference the exact basic-sequence step it extends, separate pickup collection from XP progression, and treat the level-up draft as a paused state rather than as ordinary gameplay.

The numeric acceptance bounds in Section 4 are proposed engineering targets for this feature and should be adjusted if the project's existing performance requirements or instructor rubric specifies different limits. The XP carry-over policy, drop weights, upgrade rarity rules, and persistent-loot policy must be confirmed against the game's authoritative design document before implementation.

#	Question / Request to AI	How the AI Response Was Used
1	"Using the sample document produce a champion document for the following feature: F4 — Pickups, Experience & the Upgrade Draft."	Used the provided sample Champion document as the structural model for the F4 Champion document. The response established the introduction, scenarios, data-flow section, acceptance tests, and development timeline.
2	"It needs case diagrams such as this."	Added UML-style use-case diagrams modeled after the supplied example. These diagrams show actors, system boundaries, primary use cases, <<includes>>, and <<extends>> relationships.
3	"Which classes will my dynamic binding be in?"	Used to determine where polymorphism could occur in the implementation. The proposed design used a base Pickup type with specialized classes such as XPPickup, ShardPickup, and HealthPickup.
4	"Is 'FunctionNameHere' going to be a child of Unity MonoBehaviour as that would handle dynamic binding which we need to do manually?"	Clarified that MonoBehaviour provides Unity component functionality but does not itself provide the required dynamic binding. Dynamic binding would instead be implemented through C# inheritance, abstract/virtual methods, overrides, or interfaces.
5	"Which classes are involved in my design pattern? What do the class diagrams of each design pattern look like for my feature?"	Explored possible Factory, Strategy, Observer, and State patterns for F4. This helped establish relationships among Pickup, PickupFactory, XPProgression, IXPCurve, level-up events, the upgrade draft, and gameplay states.

6	“What structure and design methodologies does the champion document follow?”	Used to identify the document as primarily structured, requirements-driven software design using use-case analysis, functional decomposition, DFDs, acceptance testing, separation of concerns, and dependency-based project planning.
7	“Based on this document generate a sequence diagram that includes at least three classes.”	Used to develop the interaction model between classes involved in the reward loop and to show the chronological flow between pickup collection, XP progression, and the upgrade system.
8	“Create a class diagram based on the Champion document produced.”	Used to translate the functional design into an object-oriented class structure for the F4 implementation.
9	“Generate a fine grained time requirements to complete this project.”	Expanded the original high-level timeline into smaller implementation tasks with durations and dependencies for pickup logic, XP progression, the draft, run loot, testing, documentation, and UI work.
10	“Fill this out.”	Used the supplied Champion template to reorganize the F4 material into the required submission format rather than leaving it as a separate design document.
11	“Add data flow diagrams to both documents.”	Added DFDs showing the overall F4 reward-loop context and the internal processes: Resolve Drop, Collect Pickup, Apply XP Curve, Run Upgrade Draft, and Finalize Run Loot.
12	“The name of the game is <i>I Got Transported to Another World and Got Married to a Vampire Shrimp But His Vampire Family Doesn't Approve Of Our Marriage</i> , not Ember Cavern.”	Corrected an inaccurate working game title that had been introduced during drafting. The correct title was then used for subsequent work.
13	“Remake the documents with this new name.”	Regenerated the Champion documents so that the corrected game title appeared in the documents and their diagrams.

- | | | |
|----|---|---|
| 14 | “Add a context diagram. Add pseudo-code for the reward loop.” | Added a context diagram defining the boundary between F4 and neighboring systems. Also added pseudocode describing drop resolution, pickup collection, magnet behavior, XP progression, queued level-ups, upgrade drafting, upgrade application, and run-loot finalization. |
| 15 | “Make an AI question log from this session.” | Produced this log summarizing the AI questions, requests, and how the responses contributed to the Champion document. |